第6章: プロジェクトのセットアップ（ナビゲーション）

> ここから実際に「書籍管理アプリ」を組み立てていきます。この章では、アプリ全体の**画面構成（ナビゲーション）**を作り、本の「型」を定義し、Supabaseとデータをやり取りする関数をまとめます。第7・8章のCRUD実装の土台になる、大切な準備の章です。

---

1. この章で作るもの（全体像）

これから作るファイルと、その役割を先に示します。

コピー

```
my-books-app/
├── app/
│   ├── _layout.tsx          ← アプリ全体の枠組み（Stackナビゲーション）
│   ├── index.tsx            ← 書籍一覧画面（第7章で中身を作る）
│   ├── new.tsx              ← 新規登録画面（第7章）
│   └── books/
│       └── [id].tsx         ← 編集画面（第8章。idで対象の本を識別）
├── lib/
│   ├── supabase.ts          ← Supabase接続（第5章で作成済み）
│   └── books.ts             ← 本のCRUD関数をまとめる（この章で作る）
└── types/
    └── book.ts              ← 本の「型（Book型）」を定義（この章で作る）
```

> **本書のナビゲーション方針:** 第4章で「Tabs（下タブ）」にも触れましたが、書籍管理アプリはシンプルに「一覧 → 登録／編集」と画面を重ねる **Stack ナビゲーション** で作ります。Expoテンプレートに最初から入っている `(tabs)` フォルダは使わない構成に整理します。

---

2. 本の「型」を定義する

第2章で学んだ `type` を使い、アプリ全体で使う「本のデータの形」を1か所に定義します。こうすると、どのファイルでも同じ型を使えて安全です。

プロジェクト内に `types` フォルダを作り、`book.ts` を新規作成します。

▼ このコードがやること（先に日本語で）：アプリ全体で使う「本のデータの形（型）」を1か所にまとめて定義します。`Book` は「すでに保存済みの本」を表す型、`NewBook` は「これから登録する本」を表す型で、IDや登録日時の有無で2つに分けています。こうして型を1か所に置いておくと、どの画面でも同じ形を使い回せて、入力ミスをエディタが早めに教えてくれます（各項目の意味はコード内のコメントで説明します）。

コピー

```
// types/book.ts – アプリ全体で使う「本」の型を定義する

// Book : 1冊の本を表す型。Supabaseのbooksテーブルの列と対応させる（第5章で作った表）
export type Book = {
  // 本のID（Supabaseが自動生成するuuid）
  id: string;
  // タイトル（必須）
  title: string;
  // 著者（必須）
  author: string;
  // 読書状態（"未読" / "読書中" / "読了"）
  status: string;
  // メモ（空のこともあるので string または null）。第2章の「| null」参照
  memo: string | null;
  // 登録日時（Supabaseが自動で入れる）
  created_at: string;
};

// NewBook : 「新規登録するときの入力データ」の型
// id や created_at はSupabaseが自動で付けるので、入力時には不要。だからそれらを除いた形にする
export type NewBook = {
  title: string;
  author: string;
  status: string;
  memo: string | null;
};
```

`string | null` の意味（復習）：`|`（パイプ、縦棒）は「または」を表し、「`string` 型 または `null`」という意味です。これを「ユニオン型（union type）」と呼びます。memoは入力されないこともあるので、文字列かnullのどちらかを許す型にしています。

なぜ `Book` と `NewBook` を分ける？ 既存の本（`Book`）はIDや登録日時を持っていますが、これから**新しく作る**本（`NewBook`）にはまだIDがありません（Supabaseが保存時に発番するため）。入力時に余計な項目を要求しないよう、型を分けておくと安全で分かりやすくなります。

▼ コードを1つずつ分解して解説

上の型定義を、2つの塊に分けて、初心者向けにていねいに見ていきましょう。

解説1: 保存済みの本を表す `Book` 型

コピー

```
export type Book = {
  // 本のID（Supabaseが自動生成するuuid）
  id: string;
  // タイトル（必須）
  title: string;
  // 著者（必須）
  author: string;
  // 読書状態（"未読" / "読書中" / "読了"）
  status: string;
  // メモ（空のこともあるので string または null）。第2章の「| null」参照
  memo: string | null;
  // 登録日時（Supabaseが自動で入れる）
  created_at: string;
};
```

- `export`（エクスポート）は「この型を**他のファイルからも使えるように公開する**」という宣言です。これを付けることで、画面側のファイルから `import { Book }` の形で借りられます。
- `type Book = { ... }` は「`Book` という名前で、`{ }` の中の形のデータを表す型を作る」という意味です。1冊の本が持つべき項目（id・title・author・status・memo・created_at）を1か所に列挙しています。
- 各項目の `: string` などは「その項目の値の型」です。`id` や `title` は必ず文字列が入る、という約束を表します。
- `memo` だけ `string | null` になっているのは「メモは**書かれていないこともある**」ためです（後述の用語補足参照）。
- `id` と `created_at` は**Supabaseが自動で付ける**項目なので、利用者が手で入れる必要はありません。

> **用語:** `export`（エクスポート）… ファイルの中で作った型・関数・変数を「外部に公開」する印。逆に、別ファイルでそれを受け取るのが `import`（インポート）です。この2つはセットで「ファイルをまたいで部品を共有する」仕組みになります。

解説2: 新規登録用の `NewBook` 型

コピー

```
export type NewBook = {
  title: string;
  author: string;
  status: string;
  memo: string | null;
};
```

- `NewBook` は「**これから登録する本の入力データ**」だけを表す型です。`Book` から `id` と `created_at` を取り除いた4項目だけになっています。
- なぜ取り除くかというと、`id`（本のID）と `created_at`（登録日時）は**Supabaseが保存時に自動で発番・記録する**ため、登録する人が用意する必要がないからです。
- 入力に必要な項目だけに絞ることで、「登録フォームでは何を入力すればよいか」がこの型を見るだけで分かります。
- `Book` と `NewBook` を分けておくと、「保存済みの本」と「これから作る本」をTypeScriptが別物として区別してくれ、取り違えのミスを防げます。

> **用語:** ユニオン型（union type）… `string | null` のように `|`（縦棒）で複数の型をつないで「このどれかが入る」と表す型。ここでは「文字列が入っているか、あるいは何も無い（null）か」のどちらかを許しています。

---

3. CRUD関数をまとめる — `lib/books.ts`

Supabaseとのデータのやり取り（取得・追加・更新・削除）を、各画面に散らばらせず**1つのファイルにまとめます**。こうすると、画面側のコードがすっきりし、後から修正も楽になります。

`lib` フォルダに `books.ts` を新規作成します。第7・8章で各関数を実際に使います。

> **▼ このコードがやること（先に日本語で）**：Supabase（データの保管庫）と本のデータをやり取りする関数を、まとめて1つのファイルに用意します。「全件取得」「1件追加」「1件取得」「更新」「削除」という、いわゆる **CRUD（作成・読取・更新・削除）** の5つの関数を作ります。ポイントは、画面のコードからは `fetchBooks()` のように関数を呼ぶだけで済むようにし、データベースとのやり取りの細かい書き方をこのファイルに閉じ込めることです（各関数の中身はコード内のコメントで説明します）。

コピー

```
// lib/books.ts – 本のCRUD（作成・読取・更新・削除）をまとめたファイル

// 第5章で作った接続オブジェクトを借りる
import { supabase } from "./supabase";
// さっき作った型を借りる
import { Book, NewBook } from "../types/book";

// ============================================================
// Read（読み取り）：本を全件取得する
// ============================================================
// async ：中で await（待つ処理）を使うので付ける（第2章参照）
// : Promise<Book[]> ：この関数は「最終的にBookの配列を返す」という戻り値の型
export async function fetchBooks(): Promise<Book[]> {
  // supabase.from("books") ：booksテーブルを対象にする
  // .select("*")           ：全列(*)を取得する
  // .order("created_at", { ascending: false }) ：created_atで並べ替え。降順（新しい順）
  const { data, error } = await supabase
    .from("books")
    .select("*")
    .order("created_at", { ascending: false });

  // エラーがあれば throw で呼び出し元に知らせる（第7章で受け止める）
  if (error) throw error;
  // データを返す。?? は「左がnull/undefinedなら右を使う」。空なら[]を返す
  return data ?? [];
}

// ============================================================
// Create（作成）：本を1冊追加する
// ============================================================
// 引数 newBook ：登録する本のデータ（NewBook型）
export async function createBook(newBook: NewBook): Promise<void> {
  // .insert(newBook) ：受け取ったデータをbooksテーブルに追加する
  const { error } = await supabase.from("books").insert(newBook);
  if (error) throw error;
}

// ============================================================
// Read（1件）：idを指定して1冊だけ取得する（編集画面で使う）
// ============================================================
export async function fetchBookById(id: string): Promise<Book> {
  const { data, error } = await supabase
    .from("books")
    .select("*")
    // .eq("id", id) ：id列が引数idと等しい行に絞り込む（eq = equal）
    .eq("id", id)
    // .single()     ：結果を「1件のオブジェクト」として受け取る（配列でなく）
    .single();

  if (error) throw error;
```

```
  return data;
}

// ============================================================
// Update（更新）：idの本を新しい内容に書き換える
// ============================================================
export async function updateBook(id: string, updated: NewBook): Promise<void> {
  const { error } = await supabase
    .from("books")
    // .update(updated) ：指定データで上書きする
    .update(updated)
    // .eq("id", id)    ：id列が一致する行だけを更新（これが無いと全件更新になり危険）
    .eq("id", id);
  if (error) throw error;
}

// ============================================================
// Delete（削除）：idの本を削除する
// ============================================================
export async function deleteBook(id: string): Promise<void> {
  const { error } = await supabase
    .from("books")
    // .delete() ：行を削除する
    .delete()
    // .eq("id", id) ：id列が一致する行だけを削除
    .eq("id", id);
  if (error) throw error;
}
```

> **Supabaseのメソッドチェーン**：`supabase.from("books").select("*").order(...)` のように、`.` でつないで条件を足していく書き方を「メソッドチェーン」と呼びます。「booksテーブルから → 全列を選んで → 並べ替える」と、左から右へ読めば処理が分かります。
>
> **`throw error` の意味**：`throw`（スロー）は「エラーを呼び出し元に投げて知らせる」命令です。ここでエラーを投げておくと、画面側で `try / catch`（第7章で説明）を使って「エラーなら画面にメッセージを出す」といった対応ができます。
>
> **`?? []`（ヌル合体演算子）**：`data ?? []` は「`data` が null か undefined なら、代わりに空配列 `[]` を使う」という意味です。「データが取れなかったときも、せめて空のリストを返す」ことで、画面側がエラーになりにくくなります。

▼ コードを1つずつ分解して解説

このファイルは5つの関数の集まりです。まずファイル先頭の「借り物（import）」を見たあと、関数を1つずつ分解していきます。

解説1: 他ファイルから道具を借りる（import）

コピー

```
// 第5章で作った接続オブジェクトを借りる
import { supabase } from "./supabase";
// さっき作った型を借りる
import { Book, NewBook } from "../types/book";
```

- `import { 名前 } from "場所"` は「**別のファイルで `export` した部品を、このファイルに借りてくる**」書き方です。解説した `export` の受け取り側にあたります。
- 1行目で、第5章で用意した `supabase`（データベースとの接続を担当するオブジェクト）を借りています。`"./supabase"` の `./` は「同じ `lib` フォルダの中」という意味です。
- 2行目で、先ほど定義した `Book` 型と `NewBook` 型を借りています。`"../types/book"` の `../` は「1つ上のフォルダへ戻ってから `types` フォルダへ」という意味です。
- これらを借りることで、以下の各関数の中で `supabase.from(...)` を呼んだり、`Book` 型を戻り値に指定したりできます。

> **用語:** 相対パス（relative path）… `./` は「今のフォルダ」、`../` は「1つ上のフォルダ」を表す道案内の書き方。`import` でファイルの場所を指定するときに使います。

解説2: 全件取得する `fetchBooks`

コピー

```
export async function fetchBooks(): Promise<Book[]> {
  const { data, error } = await supabase
    .from("books")
    .select("*")
    .order("created_at", { ascending: false });

  // エラーがあれば throw で呼び出し元に知らせる（第7章で受け止める）
  if (error) throw error;
  // データを返す。?? は「左がnull/undefinedなら右を使う」。空なら[]を返す
  return data ?? [];
}
```

- `async function fetchBooks()` は「**非同期で動く関数**」です。`async` を付けると、中で `await`（処理が終わるのを待つ）を使えます。
- 戻り値の型 `Promise<Book[]>` は「最終的に `Book` の配列（複数の本）を返す」という意味です。`Promise`（プロミス）は「いま結果は無いが、あとで結果が届く」ことを表す入れ物です。
- `await supabase.from("books").select("*").order(...)` で「booksテーブルから全列を取り、登録日時の新しい順に並べる」という問い合わせを実行し、終わるまで待ちます。

- 結果は `{ data, error }` の形で返るので分割代入で取り出し、`error` があれば `throw` で中断、無ければ `data ?? []` を返します。

> **用語: `async` / `await`**（エイシンク／アウェイト）… 時間のかかる処理（DBアクセスや通信）を「終わるまで待ってから次へ進む」ように書くための書き方。`await` は `async` の付いた関数の中でしか使えません。

解説3: 1冊追加する `createBook`

コピー

```
export async function createBook(newBook: NewBook): Promise<void> {
  // .insert(newBook) : 受け取ったデータをbooksテーブルに追加する
  const { error } = await supabase.from("books").insert(newBook);
  if (error) throw error;
}
```

- 引数 `newBook: NewBook` は「これから登録する本のデータ」で、型は先ほどの `NewBook`（id・登録日時を含まない4項目）です。
- 戻り値の型 `Promise<void>` の `void`（ボイド）は「**返す値は特に無い**」という意味です。追加するだけで、呼び出し元に渡す結果は無いためこうしています。
- `.insert(newBook)` で「受け取った `newBook` を booksテーブルに新しい行として追加」します。`id` と `created_at` はSupabase側が自動で付けます。
- 取得が無いので分割代入は `{ error }` だけ。失敗していれば `throw error` で呼び出し元に知らせます。

> **用語: `void`**（ボイド）… 関数の戻り値が「無い（返さない）」ことを表す型。「処理はするが結果は受け取らなくてよい」関数に付けます。

解説4: idで1冊だけ取得する `fetchBookById`

コピー

```
export async function fetchBookById(id: string): Promise<Book> {
  const { data, error } = await supabase
    .from("books")
    .select("*")
    // .eq("id", id) : id列が引数idと等しい行に絞り込む（eq = equal）
    .eq("id", id)
    // .single()     : 結果を「1件のオブジェクト」として受け取る（配列でなく）
    .single();

  if (error) throw error;
  return data;
}
```

- 引数 `id: string` で「取り出したい本のID」を受け取ります。戻り値の型 `Promise<Book>` は「`Book` 1件を返す」という意味です（配列ではなく1冊）。
- `.eq("id", id)` は「**id列の値が、引数で渡した `id` と等しい行だけに絞り込む**」条件です。`eq` は equal（等しい）の略です。
- `.single()` を付けると、結果を「配列の中の1件」ではなく「**1個のオブジェクト**」として受け取れます。編集画面で「特定の1冊」を扱うのにちょうどよい形です。
- エラーが無ければ、取り出した1件 `data` をそのまま返します。

> **用語:** `.eq()`（イコール）… Supabaseで「ある列の値が指定値と等しい行だけ」に絞り込む条件メソッド。`.eq("列名", 値)` の形で使います。

解説5: 内容を書き換える `updateBook`

コピー

```
export async function updateBook(id: string, updated: NewBook): Promise<void> {
  const { error } = await supabase
    .from("books")
    // .update(updated) : 指定データで上書きする
    .update(updated)
    // .eq("id", id)    : id列が一致する行だけを更新（これが無いと全件更新になり危険）
    .eq("id", id);
  if (error) throw error;
}
```

- 引数は2つで、`id`（どの本を直すか）と `updated`（新しい内容、`NewBook` 型）です。
- `.update(updated)` で「指定したデータの内容に**上書き**する」よう指示します。

- `.eq("id", id)` が**非常に重要**です。これで「id列が一致する1行だけ」を更新対象に絞ります。**この `.eq()` を書き忘れると、booksテーブルの全件が同じ内容に書き換わってしまう**ので、必ず付けます。
- 戻り値は `Promise<void>`（返す値は無し）。失敗していれば `throw error` で知らせます。

> **用語:** `.update()`（アップデート）… Supabaseで「既存の行の内容を新しい値で上書き」するメソッド。多くの場合 `.eq()` とセットで「どの行を更新するか」を必ず指定します。

解説6: 削除する `deleteBook`

コピー

```
export async function deleteBook(id: string): Promise<void> {
  const { error } = await supabase
    .from("books")
    // .delete() : 行を削除する
    .delete()
    // .eq("id", id) : id列が一致する行だけを削除
    .eq("id", id);
  if (error) throw error;
}
```

- 引数 `id` で「削除したい本のID」を受け取ります。戻り値は `Promise<void>`（返す値は無し）です。
- `.delete()` は「行を削除する」メソッドです。引数は不要で、次の `.eq()` で対象を指定します。
- `updateBook` と同様に、`.eq("id", id)` で「id列が一致する1行だけ」に絞ります。**これが無いと全件削除になってしまう**ため、削除でも必ず付けます。
- 失敗していれば `throw error` で呼び出し元に知らせ、画面側で対応できるようにします。

> **用語:** `.delete()`（デリート）… Supabaseで「行を削除する」メソッド。`.update()` と同じく、`.eq()` で対象行を限定しないと全件が消えてしまうため注意が必要です。

---

4. ナビゲーションの枠組みを作る — `_layout.tsx`

アプリ全体の画面の重ね方（Stack）と、各画面のヘッダー（上部バー）のタイトルを設定します。`app/_layout.tsx` を次の内容にします（既存内容は置き換え）。

▼ **このコードがやること（先に日本語で）**：アプリ全体の「画面の重ね方」と「上部バー（ヘッダー）の見た目」をまとめて設定します。`Stack` は画面をカードのように積み重ねて遷移させる仕組みで、共通の設定（色など）は全画面まとめて、タイトルは画面ごとに指定します。`_layout.tsx` はアプリの土台になる特別なファイルで、ここで一度決めておけば各画面に同じ枠組みが自動で適用されます（細かい指定はコード内のコメントで説明します）。

コピー

```
// app/_layout.tsx – アプリ全体のナビゲーションの枠組み

// Stack（カードを重ねる遷移）を借りる
import { Stack } from "expo-router";

export default function RootLayout() {
  return (
    // Stack : 画面を積み重ねるナビゲーション。screenOptions で全画面共通の見た目を指定
    <Stack
      screenOptions={{
        // ヘッダー（上部バー）の背景色を青に
        headerStyle: { backgroundColor: "#1e40af" },
        // ヘッダーの文字・戻る矢印の色を白に
        headerTintColor: "#fff",
        // ヘッダータイトルを太字に
        headerTitleStyle: { fontWeight: "bold" },
      }}
    >
      {/* Stack.Screen : 各画面ごとの個別設定。name はファイル名（拡張子なし）と対応 */}
      <Stack.Screen name="index" options={{ title: "📚 書籍管理" }} />
      {/* index.tsx → タイトル「書籍管理」。これが最初に表示される画面 */}

      <Stack.Screen name="new" options={{ title: "新規登録" }} />
      {/* new.tsx → タイトル「新規登録」 */}

      <Stack.Screen name="books/[id]" options={{ title: "編集" }} />
      {/* books/[id].tsx → タイトル「編集」。[id]は動的な値（第4章参照） */}
    </Stack>
  );
}
```

`screenOptions` と `options` の違い：- `screenOptions`（Stack全体に付ける）… **すべての画面に共通**する設定。- `options`（各 `Stack.Screen` に付ける）… その**画面だけ**の設定（タイトルなど）。こうして「色は全画面共通、タイトルは画面ごと」と整理できます。

`name` はファイル名と一致させる：`<Stack.Screen name="books/[id]" ... />` の `name` は、`app/` フォルダからの**ファイルのパス（拡張子なし）**と一致させます。`app/books/[id].tsx` なら `name="books/[id]"` です。一致していないと設定が効きません。

▼ コードを1つずつ分解して解説

この土台ファイルを、3つの塊に分けて見ていきましょう。

解説1: 道具を借りて、土台コンポーネントを宣言する

コピー

```
// Stack（カードを重ねる遷移）を借りる
import { Stack } from "expo-router";

export default function RootLayout() {
```

- `import { Stack } from "expo-router"` で、画面をカードのように積み重ねて遷移させる `Stack` という部品を、`expo-router`（ルーティング用ライブラリ）から借りています。
- `export default function RootLayout()` は「`RootLayout` という名前の土台コンポーネントを作り、**このファイルの代表として公開する**」という宣言です。
- `_layout.tsx` という名前のファイルは expo-router にとって特別で、**そのフォルダ全体の枠組み**を表します。ここで一度決めた設定が、配下の各画面に自動で適用されます。
- `default`（デフォルト）が付くと「このファイルの主役は1つだけ」という公開のしかたになり、利用側は名前を自由に付けて借りられます。

> **用語:** `export default`（エクスポート・デフォルト）… 1ファイルにつき1つだけ指定できる「主役の公開」。画面ファイルは必ずこの形で画面コンポーネントを1つ公開します。

解説2: 全画面共通の見た目を `screenOptions` で指定する

コピー

```
    <Stack
      screenOptions={{
        // ヘッダー（上部バー）の背景色を青に
        headerStyle: { backgroundColor: "#1e40af" },
        // ヘッダーの文字・戻る矢印の色を白に
        headerTintColor: "#fff",
        // ヘッダータイトルを太字に
        headerTitleStyle: { fontWeight: "bold" },
      }}
    >
```

- `<Stack screenOptions={{ ... }}>` の `screenOptions` は「**すべての画面に共通する見た目の設定**」をまとめて渡す場所です。

- `headerStyle: { backgroundColor: "#1e40af" }` で、上部バー（ヘッダー）の背景色を青（`#1e40af`）にしています。`#` から始まる6桁はカラーコード（色の指定）です。
- `headerTintColor: "#fff"` は、ヘッダー上の文字や「戻る矢印」の色を白（`#fff`）にします。青い背景に白文字で見やすくする設定です。
- `headerTitleStyle: { fontWeight: "bold" }` で、ヘッダーのタイトル文字を太字にしています。
- ここで一度指定すれば、後述の各画面すべてに同じヘッダーの見た目が適用されます。

> **用語:** カラーコード（color code）… `#1e40af` のように `#` と6桁で色を表す書き方。前2桁=赤、中2桁=緑、後2桁=青の強さを16進数で表します。`#fff` は `#ffffff`（白）の短縮形です。

解説3: 画面ごとの設定を `Stack.Screen` で並べる

コピー

```
      <Stack.Screen name="index" options={{ title: "📚 書籍管理" }} />
      <Stack.Screen name="new" options={{ title: "新規登録" }} />
      <Stack.Screen name="books/[id]" options={{ title: "編集" }} />
    </Stack>
```

- `<Stack.Screen ... />` は「**1つの画面ごとの個別設定**」を表します。3行あるので、3つの画面を登録しています。
- `name="index"` などの `name` は、`app/` フォルダからの**ファイルのパス（拡張子なし）**と一致させます。`index.tsx` なら `name="index"` です。
- `options={{ title: "..." }}` で、その画面だけのヘッダータイトルを指定します。`index` は最初に開く一覧画面なので「📚 書籍管理」としています。
- `name="books/[id]"` の `[id]` は「動的な値」を表し、`app/books/[id].tsx` という編集画面に対応します（第4章参照）。

> **用語:** 動的ルート（dynamic route）… `[id]` のように `[ ]` で囲んだファイル名で「その部分が可変の値（本のIDなど）」になる仕組み。`books/123` でも `books/456` でも同じ `[id].tsx` が使われます。

4.1 不要なテンプレートファイルの整理

Expoのテンプレートには `app/(tabs)/` フォルダなどサンプル画面が入っています。本書のStack構成に合わせ、次のように整理します。

コピー

```
# ターミナルで、不要なテンプレート画面フォルダを削除する例（プロジェクト直下で実行）
# 注意: 削除は慎重に。中身を確認してから実行してください
# rm はファイル/フォルダを削除するコマンド / -r は「フォルダごと（再帰的に）」の意味
rm -r app/(tabs)
# Windowsの PowerShell では次のように書く：
# Remove-Item -Recurse -Force "app/(tabs)"
```

削除に不安があれば: 無理に消さなくても進められます。その場合は、これから作る `index.tsx` などを `app/` 直下に置けば、そちらが優先して使われます。慣れないうちは「新しいファイルを `app/` 直下に作る」方針でも構いません。整理は後からでもできます。

---

5. プレースホルダー画面を置いて動作確認

第7・8章で中身を作る前に、まず「空っぽの画面」を置いて、ナビゲーション（画面遷移）が動くことを確認します。

5.1 一覧画面（仮）

`app/index.tsx` を次の内容にします。

▼ このコードがやること（先に日本語で）: 最初に表示される「書籍一覧画面」を、まずは中身が空の仮の状態で作ります（本格的な一覧表示は第7章で実装します）。今回の目的は、画面に置いたボタンを押すと新規登録画面へ移動できることを確かめることです。`useRouter` で画面移動の道具を取り出し、`router.push("/new")` で別の画面へ進みます（各部分の意味はコード内のコメントで説明します）。

コピー

```
// app/index.tsx - 書籍一覧画面（この章では仮の内容。第7章で本実装）

import { View, Text, Pressable, StyleSheet } from "react-native";
// 画面移動の道具（第4章参照）
import { useRouter } from "expo-router";

export default function HomeScreen() {
  // 移動操作を取得
  const router = useRouter();

  return (
    <View style={styles.container}>
      <Text style={styles.text}>ここに書籍一覧が表示されます（第7章で実装）</Text>

      {/* 新規登録画面へ移動するボタン */}
      <Pressable style={styles.button} onPress={() => router.push("/new")}>
        {/* router.push("/new") : new.tsx（新規登録画面）へ移動 */}
        <Text style={styles.buttonText}>＋ 新規登録へ</Text>
      </Pressable>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: "center", alignItems: "center", gap: 16, padding: 20 },
  text: { fontSize: 14, color: "#475569", textAlign: "center" },
  button: { backgroundColor: "#1e40af", paddingVertical: 12, paddingHorizontal: 20, borderRadius: 10 },
  buttonText: { color: "#fff", fontWeight: "bold" },
});
```

▼ コードを1つずつ分解して解説

この仮の一覧画面を、4つの塊に分けて見ていきましょう。

解説1: 部品を借りて、画面移動の道具を取り出す

コピー

```
import { View, Text, Pressable, StyleSheet } from "react-native";
// 画面移動の道具（第4章参照）
import { useRouter } from "expo-router";

export default function HomeScreen() {
  // 移動操作を取得
  const router = useRouter();
```

- 1行目で `react-native` から画面部品を借りています。`View`（箱）、`Text`（文字）、`Pressable`（押せる領域）、`StyleSheet`（見た目の設定をまとめる道具）の4つです。
- 2行目の `useRouter`（ユーズ・ルーター）は「**画面移動を操作するための道具**」を取り出すフック（部品）です。

- `export default function HomeScreen()` で、この画面の主役コンポーネントを公開しています。一覧画面なので `HomeScreen` という名前です。
- `const router = useRouter()` で、移動操作をまとめた `router` を受け取ります。以降 `router.push(...)` の形で別画面へ進めます。

> **用語:** フック（hook）… `useRouter` のように `use` で始まる、Reactの機能を呼び出す関数。コンポーネントの中で呼んで、状態や便利な道具を取り出します。

解説2: 画面の中身を `View` と `Text` で組み立てる

コピー

```
    <View style={styles.container}>
      <Text style={styles.text}>ここに書籍一覧が表示されます（第7章で実装）</Text>
```

- `<View>` はReact Nativeの「**箱**」で、Web の `<div>` にあたります。中に他の部品を入れてまとめます。
- `style={styles.container}` で、下のほうで定義した `container` の見た目（中央寄せ・余白など）を適用しています。
- `<Text>` は「文字を表示する部品」です。React Nativeでは、文字は**必ず `<Text>` の中に書く**決まりがあります（Webのように地の文を直接置けません）。
- ここでは「第7章で本実装する」という案内文を仮置きしています。

> **用語:** `View` / `Text`（ビュー／テキスト）… React Nativeの基本部品。`View` は領域をまとめる箱、`Text` は文字専用の部品。Webの `<div>` と「文字は必ずタグで包む」点が違います。

解説3: ボタンを押すと新規登録画面へ進む

コピー

```
      {/* 新規登録画面へ移動するボタン */}
      <Pressable style={styles.button} onPress={() => router.push("/new")}>
        {/* router.push("/new") : new.tsx（新規登録画面）へ移動 */}
        <Text style={styles.buttonText}>＋ 新規登録へ</Text>
      </Pressable>
```

- `<Pressable>` は「**押せる領域**」を作る部品です。中に入れた `<Text>` がボタンの文字になります。
- `onPress={() => router.push("/new")}` が押したときの動作です。`onPress` には「押されたら呼ばれる関数」を渡します。
- `() => router.push("/new")` はアロー関数で、「押された瞬間に `router.push("/new")` を実行する」という意味です。`() =>` で包むのは、**今すぐ実行せず「押されたとき」に実行させる**ためです。

- `router.push("/new")` は「`new.tsx`（新規登録画面）を上に重ねて表示する」操作です。`push`（プッシュ）は「新しい画面を積む」イメージです。

> **用語:** `router.push("/パス")`（プッシュ）… 指定したパスの画面を、現在の画面の上に新しく重ねて表示する操作。戻ると元の画面が下から現れます。

解説4: 見た目を `StyleSheet.create` でまとめる

コピー

```
const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: "center", alignItems: "center", gap: 16, padding: 20 },
  text: { fontSize: 14, color: "#475569", textAlign: "center" },
  button: { backgroundColor: "#1e40af", paddingVertical: 12, paddingHorizontal: 20, borderRadius: 10 },
  buttonText: { color: "#fff", fontWeight: "bold" },
});
```

- `StyleSheet.create({ ... })` は「**見た目の設定をまとめて作る**」道具です。作った設定は `styles.container` のように名前で呼び出して使います。
- `container` は画面全体の箱の設定です。`flex: 1`（画面いっぱいに広げる）、`justifyContent` / `alignItems: "center"`（縦横の中央寄せ）、`gap: 16`（部品の間隔）、`padding: 20`（内側の余白）を指定しています。
- `text` は案内文の設定（文字サイズ・色・中央寄せ）、`button` はボタンの設定（背景色・余白・角丸）です。
- `buttonText` はボタン内の文字の設定で、白文字・太字にしています。

> **用語:** `StyleSheet.create`（スタイルシート・クリエイト）… React Nativeで見た目をまとめて定義する仕組み。WebのCSSにあたり、`flex` や `padding` など似た指定が使えます。

5.2 新規登録画面（仮）

`app/new.tsx` を新規作成します。

> **▼ このコードがやること（先に日本語で）:** 「新規登録画面」を、こちらもまずは中身が空の仮の状態で作ります（登録フォームの本体は第7章で実装します）。狙いは、一覧画面から移動してきたあと、ボタンで元の画面（一覧）へ戻れることを確かめることです。`router.back()` は「1つ前の画面に戻る」操作で、これでStackナビゲーションの行き来が一通り確認できます（各部分の意味はコード内のコメントで説明します）。

コピー

```
// app/new.tsx – 新規登録画面（この章では仮の内容。第7章で本実装）

import { View, Text, Pressable, StyleSheet } from "react-native";
import { useRouter } from "expo-router";

export default function NewBookScreen() {
  const router = useRouter();

  return (
    <View style={styles.container}>
      <Text style={styles.text}>ここに登録フォームが表示されます（第7章で実装）</Text>

      {/* 前の画面（一覧）へ戻るボタン */}
      <Pressable style={styles.button} onPress={() => router.back()}>
        {/* router.back() : 1つ前の画面に戻る */}
        <Text style={styles.buttonText}>戻る</Text>
      </Pressable>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: "center", alignItems: "center", gap: 16, padding: 20 },
  text: { fontSize: 14, color: "#475569", textAlign: "center" },
  button: { backgroundColor: "#64748b", paddingVertical: 12, paddingHorizontal: 20, borderRadius: 10 },
  buttonText: { color: "#fff", fontWeight: "bold" },
});
```

▼ コードを1つずつ分解して解説

この仮の新規登録画面は、一覧画面とよく似ています。違いが出る「戻る」部分を中心に、3つの塊で見ていきましょう。

解説1: 部品を借りて、画面を宣言する

コピー

```
import { View, Text, Pressable, StyleSheet } from "react-native";
import { useRouter } from "expo-router";

export default function NewBookScreen() {
  const router = useRouter();
```

- 借りている部品（`View` / `Text` / `Pressable` / `StyleSheet`）と `useRouter` は、一覧画面とまったく同じです。
- `export default function NewBookScreen()` で、この画面の主役を公開しています。新規登録画面なので `NewBookScreen` という名前にしています。
- `const router = useRouter()` で画面移動の道具 `router` を受け取ります。この画面では「戻る」操作に使います。

- このように、画面ファイルは「部品を借りる → 画面コンポーネントを公開する → 必要な道具を取り出す」という同じ骨組みを持ちます。

> **用語:** コンポーネント名（PascalCase）… `NewBookScreen` のように、画面や部品の名前は単語の先頭を大文字にする習慣です。Reactはこの大文字始まりを「部品」と判断します。

解説2: 案内文と「戻る」ボタンを並べる

コピー

```
    <View style={styles.container}>
      <Text style={styles.text}>ここに登録フォームが表示されます（第7章で実装）</Text>

      {/* 前の画面（一覧）へ戻るボタン */}
      <Pressable style={styles.button} onPress={() => router.back()}>
        {/* router.back() : 1つ前の画面に戻る */}
        <Text style={styles.buttonText}>戻る</Text>
      </Pressable>
```

- `<View>` で全体を包み、中に案内文の `<Text>` と「戻る」ボタンの `<Pressable>` を並べています。
- `onPress={() => router.back()}` が、一覧画面の `push` と対になる部分です。`router.back()` は「**1つ前の画面に戻る**」操作です。
- 一覧画面で `router.push("/new")` でこの画面を上に積んだので、`router.back()` でその1枚を取り除き、下にあった一覧画面に戻ります。
- `() =>` で包んでいるのは一覧画面と同じく、「押された瞬間」に実行させるためです。

> **用語:** `router.back()`（バック）… 積み重なった画面を1枚めくって「1つ前の画面」に戻る操作。`push` で進み、`back` で戻る、という行き来でStackナビゲーションが成り立ちます。

解説3: 見た目を定義する（戻るボタンは灰色）

コピー

```
const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: "center", alignItems: "center", gap: 16, padding: 20 },
  text: { fontSize: 14, color: "#475569", textAlign: "center" },
  button: { backgroundColor: "#64748b", paddingVertical: 12, paddingHorizontal: 20, borderRadius: 10 },
  buttonText: { color: "#fff", fontWeight: "bold" },
});
```

- `container` と `text` は一覧画面と同じ設定で、中央寄せ・余白・文字の見た目を指定しています。
- 違いは `button` の `backgroundColor` です。一覧画面は青（`#1e40af`）でしたが、こちらは灰色（`#64748b`）にしています。

- これは「進む（目立つ青）／戻る（控えめな灰色）」と色で役割を分け、ユーザーが直感的に区別できるようにする工夫です。
- `buttonText` は一覧画面と同じく、白文字・太字でボタン文字を見やすくしています。

> **用語:** `backgroundColor` （バックグラウンドカラー）… 部品の背景色を指定するスタイル。ここでは進むボタンと戻るボタンで色を変え、操作の意味を見た目で伝えています。

5.3 動作確認

`npx expo start` でアプリを起動し、スマホ（Expo Go）で確認します。

1. 最初に「書籍管理」というタイトルの一覧画面（仮）が表示される。
2. 「＋ 新規登録へ」を押すと、「新規登録」画面に**スライドして遷移**する。
3. ヘッダー左の「戻る矢印」または「戻る」ボタンで一覧に戻れる。

これが確認できれば、ナビゲーションの土台は完成です。Stackナビゲーションが「カードを重ねて・めくる」動きをしているのが分かるはずです。

> **うまく動かないときは:** - ヘッダーが青くならない → `_layout.tsx` の保存を確認。 - 「Unmatched Route」と出る → ファイル名（ `new.tsx` など）とパス（ `/new` ）が一致しているか確認。 - 画面が真っ白 → ターミナルのエラーを確認。多くは `import` のパスミス。

---

6. この章のまとめ

- アプリ全体で使う `Book` / `NewBook` 型を `types/book.ts` に定義した
- Supabaseとのやり取り（**fetchBooks / createBook / fetchBookById / updateBook / deleteBook**）を `lib/books.ts` に集約した
- `_layout.tsx` で Stack ナビゲーションとヘッダーの見た目を設定した
- 仮の **一覧画面・新規登録画面** を置き、画面遷移（push / back）が動くことを確認した

> **次の章へ:** 土台ができました。第7章では、この `lib/books.ts` の関数を使って、**実際に本の一覧を表示し、新しい本を登録する**機能（CRUDのReadとCreate）を実装します。